



LIONSDEV

Curso de Programação

MÓDULO 7

APIs REST com Express



Professor Nicolas Cardoso Motta

Revisando o Módulo 6



CRUD

Create, Read, Update e Delete: as quatro operações de qualquer cadastro.



ID único

O id identifica cada registro; atualizar e remover começam por encontrá-lo.



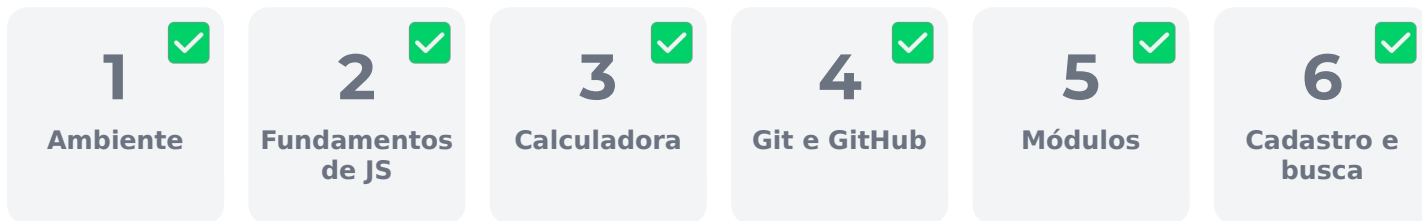
Busca e relações

find, **filter** e **some** buscam; a relação guarda o id do outro registro.

Pergunta para abrir: como outra pessoa, em outro computador, usaria o seu cadastro de contatos?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND

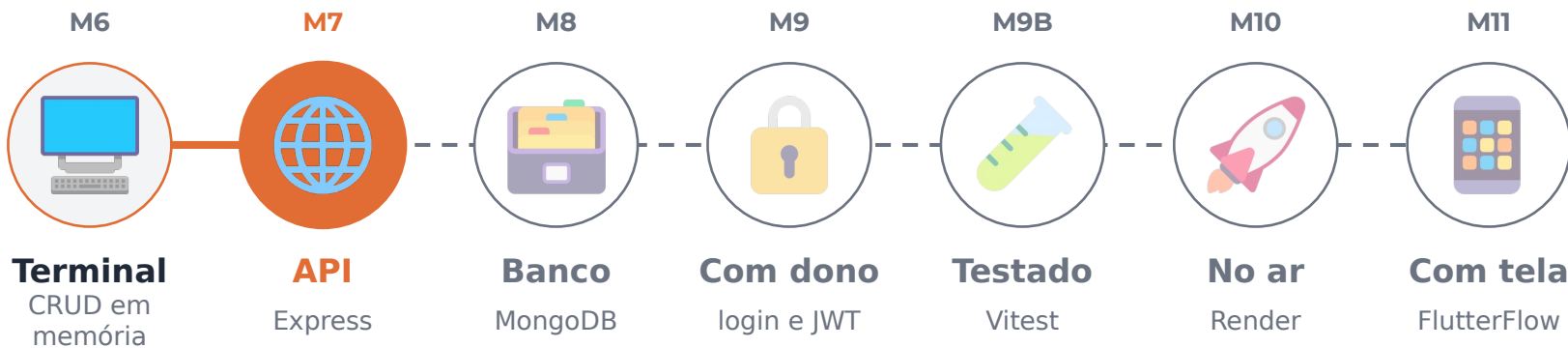


FRONTEND E PROJETO



Você está aqui: **Módulo 7, APIs REST com Express**

O projeto flashcards



NO MÓDULO 7

O CRUD do terminal vira uma API REST: cada função vira uma rota (GET /baralhos, POST /flashcards...), testada no Postman. Os dados ainda ficam na memória.



Objetivo

- **Compreender a arquitetura cliente-servidor.**
- **Entender o protocolo HTTP: métodos, status, headers e body.**
- **Saber o que é uma API REST e como organizar as rotas.**
- **Criar um servidor com Express: rotas, middlewares, validação e `express.Router()`.**
- **Testar a API com Postman, Insomnia ou cURL e transformar os flashcards numa API.**

Lembra do sistema de flashcards e da agenda do Módulo 6? Quais as **limitações de uma aplicação que roda só no terminal?**

Limitações de uma aplicação local



Acesso só onde ela roda

Só quem está naquele computador consegue usar.



Pouco intuitiva

Menus no terminal não servem para o usuário comum.



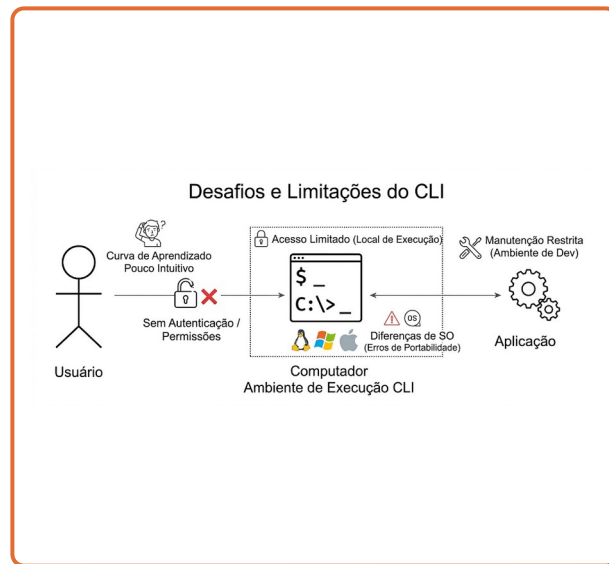
Sem autenticação

Não há login nem controle de quem pode fazer o quê.



Manutenção limitada

Cada computador tem a sua cópia: atualizar é um problema.





Cliente-Servidor

A solução para essas limitações

Quem pede e quem entrega



Cliente

O dispositivo ou software que solicita dados, páginas ou serviços: o navegador, um app.



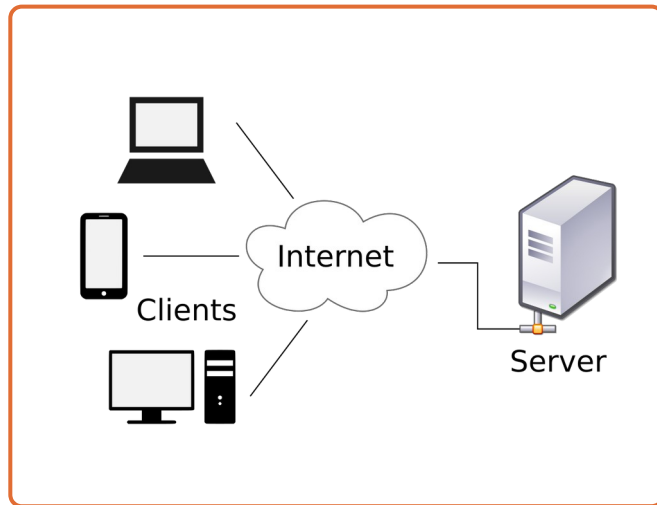
Servidor

O dispositivo ou software que fornece os dados e serviços para os clientes.



Request e response

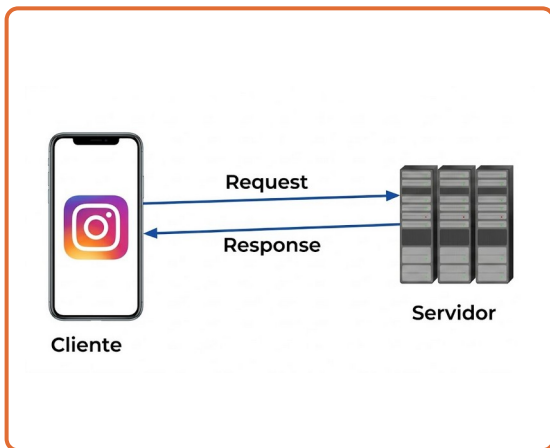
O cliente envia uma requisição (request) e o servidor devolve uma resposta (response).



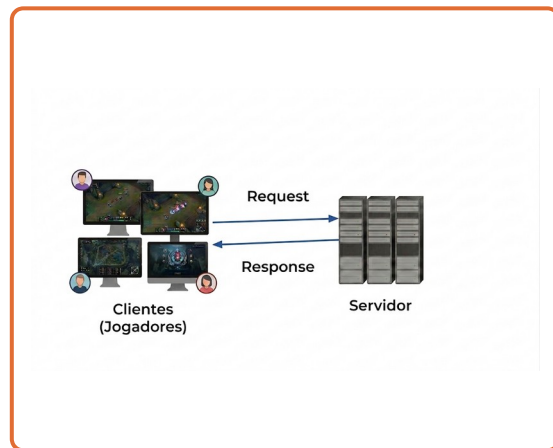
O mesmo ciclo em todo lugar



Sites: o navegador pede a página



Apps: o Instagram pede o feed



Jogos: o servidor sincroniza os jogadores

Cliente envia a request → servidor processa → devolve a response.

Vantagens e desvantagens



Vantagens

- **Centralização:** dados organizados num só lugar.
- **Escalabilidade:** muitos clientes usando o mesmo servidor.
- **Segurança:** controle de quem pode ler, criar, alterar ou remover.



Desvantagens

- **Dependência:** servidor fora do ar, ninguém acessa.
- **Latência:** a resposta depende da internet e da carga do servidor.
- **Custo:** hospedagem, domínio, banco de dados...



Protocolo HTTP

O que é HTTP?



A língua da web

HyperText Transfer Protocol: o protocolo que cliente e servidor usam para trocar dados.



Métodos

Dizem a ação: ler, criar, atualizar ou remover.



Códigos de status

Dizem o resultado: deu certo, o cliente errou ou o servidor falhou.



Headers e body

Informações extras da mensagem e os dados em si.

Estrutura de uma URL

`http://lions.dev:3000/lessons?modulo=8`

Protocolo

Domínio

Porta

Caminho

Query

Durante o curso, o domínio vai ser `localhost`: o seu próprio computador.

Ex.: `http://localhost:3000/baralhos`

Métodos HTTP

GET

Ler

Buscar dados.

POST

Criar

**Cadastrar um
novo recurso.**

PUT

Atualizar

**Alterar um
recurso que
existe.**

DELETE

Remover

**Apagar um
recurso.**

É o CRUD do Módulo 6, agora pela rede. Existe também o PATCH, para atualizar só uma parte do recurso.

Códigos de status

2xx · Sucesso

200 OK
201 Created
204 No Content

4xx · Erro do cliente

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict

5xx · Erro do servidor

500 Internal Server Error
503 Service Unavailable

O status diz de quem é o problema: 4xx, o pedido veio errado; 5xx, o servidor quebrou.

Headers e body

Requisição

```
POST /baralhos HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{ "titulo": "JavaScript" }
```

Resposta

```
HTTP/1.1 201 Created
Content-Type: application/json

{ "id": 3, "titulo": "JavaScript" }
```

Os headers trazem informações extras, como o tipo do conteúdo e a autenticação. **O body** traz os dados, em JSON numa API REST.



O que são APIs REST?

API REST

Uma forma de organizar a comunicação entre cliente e servidor usando recursos, rotas e métodos HTTP.



Recurso

A informação que a API manipula: baralhos, flashcards, contatos.



Endpoint

O endereço que dá acesso ao recurso, como `/baralhos`.



Método HTTP

A ação sobre o recurso. O mesmo endpoint faz coisas diferentes com métodos diferentes.

Um recurso, quatro ações

Método e rota	O que faz	Status de sucesso
GET /baralhos	Lista os baralhos	200 OK
GET /baralhos/:id	Busca um baralho	200 OK
POST /baralhos	Cria um baralho	201 Created
PUT /baralhos/:id	Atualiza um baralho	200 OK
DELETE /baralhos/:id	Remove um baralho	200 ou 204

O recurso vai no plural e o :id indica qual item. A ação fica por conta do método.

Frontend e backend



Frontend (cliente)

A parte visual: telas, botões e formulários. Envia requisições para o servidor.



Backend (servidor)

Recebe as requisições, aplica as regras, manipula os dados e devolve a resposta. É o que vamos construir.

O frontend chega no Módulo 11. Até lá, quem faz o papel do cliente é o Postman.



Criando uma API com Express

O que é o Express?



Um framework para Node.js

Facilita criar servidores web e APIs, instalado pelo npm.



Rotas simples

Liga um método e um caminho a uma função, em uma linha.



Construído sobre o http

O módulo nativo do Módulo 5, com muito menos trabalho.



Instalação

```
$ mkdir hello-express  
$ cd hello-express  
$ npm init -y  
$ npm install express
```

`npm install express` → baixa o Express para a `node_modules`.

`"type": "module"` → adicione no `package.json`: o código usa `import`.

`.gitignore` → coloque a `node_modules/` nele antes do primeiro commit.

O primeiro servidor

```
import express from 'express'

const app = express()
const port = 3000

app.use(express.json())

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Servidor rodando na porta ${port}`)
})
```

express() → cria a aplicação. Por convenção, chamamos de **app**.

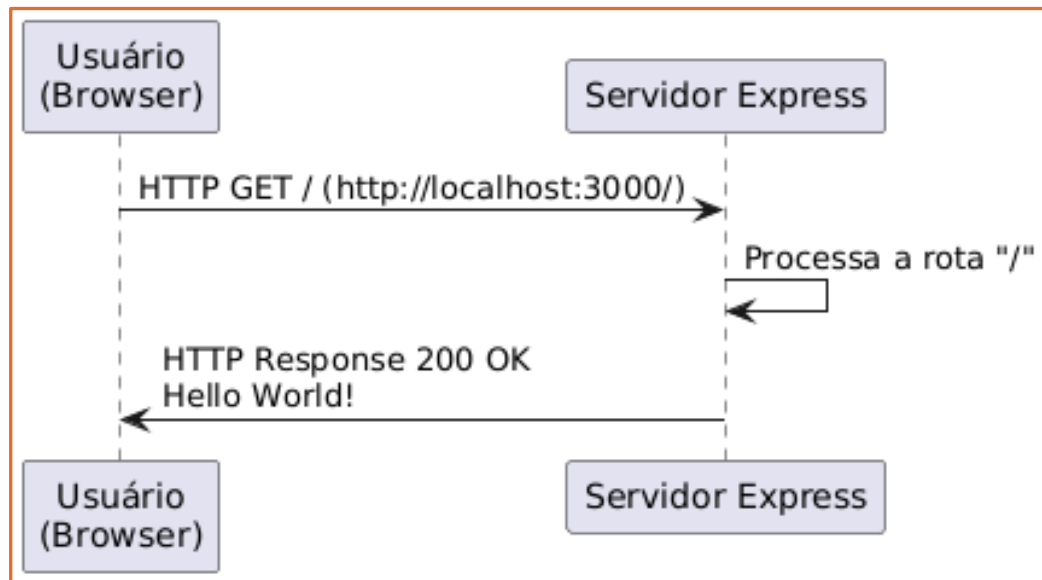
.use() → adiciona um middleware; o **express.json()** permite receber JSON.

.get() → cria uma rota: método GET no caminho **/**.

.send() → envia a resposta ao cliente.

.listen() → liga o servidor na porta, e ele fica esperando requisições.

Acessando o servidor



- Rode **node index.js**: o terminal fica "preso", porque o servidor está no ar esperando requisições.
- No navegador, acesse **http://localhost:3000**.
- O Express encontra a rota **GET /** e responde 200 OK com "Hello World!".



Cuidado: mudou o código? Reinicie o servidor

O servidor roda a versão que estava salva **quando ele foi ligado**. Mudanças não aparecem sozinhas.

- Pare com **Ctrl + C** e rode de novo, a cada alteração.
- Ou use o modo de observação do Node: ele reinicia sozinho sempre que você salva.

```
$ node --watch index.js
```



Cuidado: porta em uso

Se já existe um servidor na porta 3000, o novo não sobe e o Node mostra **EADDRINUSE**.

- Quase sempre é um servidor antigo esquecido em outro terminal: feche com **Ctrl + C**.
- Ou use outra porta, como **3001**.

```
Error: listen EADDRINUSE: address already in use :::3000
```



Testando a API

O navegador só faz GET. Para os outros métodos, usamos ferramentas.

Ferramentas de desenvolvedor



Insomnia

Visual e leve: envia requisições e mostra as respostas.



Postman

Testa, documenta e organiza as requisições. É o que vamos usar.



cURL

Pelo terminal, ótimo para testes rápidos.

Testando com cURL

```
$ curl http://localhost:3000/  
Hello World!
```

```
$ curl -X POST http://localhost:3000/baralhos \  
-H "Content-Type: application/json" \  
-d '{"titulo": "Node"}'
```

O -X define o método, o -H adiciona um header e o -d envia o body. No Postman, é o mesmo, só que com botões.



Rotas

Anatomia de uma rota

```
app.get('/baralhos', (req, res) => {  
  // lógica da rota  
  res.send('Lista de baralhos')  
})
```

app.get → o método HTTP: get, post, put ou delete.

'/baralhos' → o caminho que o cliente acessa.

req → a requisição: o que o cliente enviou.

res → a resposta: o que vamos devolver.

=> { } → a função executada quando a requisição chegar.

Várias rotas no mesmo servidor

```
app.get('/', (req, res) => {  
  res.send('Hello World!')  
})  
  
app.get('/usuarios', (req, res) => {  
  res.send('Recebendo uma requisição em /usuarios')  
})  
  
app.post('/', (req, res) => {  
  res.send('Recebendo uma requisição de POST')  
})
```

Cada combinação de método + caminho é uma rota diferente: GET / e POST / não se confundem.



Duas rotas iguais: qual responde?

```
app.get('/', (req, res) => {  
  res.send('Primeira rota!')  
})
```

```
app.get('/', (req, res) => {  
  res.send('Segunda rota!')  
})
```

Pense antes de rodar: qual é a saída?



Cuidado: a primeira rota que bate responde

Responde "**Primeira rota!**". A segunda nunca é alcançada.

- O Express testa as rotas na ordem em que foram criadas e para na primeira que combina.
- A ordem também importa com parâmetros: se `/baralhos/:id` vier antes de `/baralhos/busca`, o texto "busca" é tratado como um id.
- Regra prática: rotas fixas primeiro, rotas com parâmetro depois.



req.body, req.params e req.query

Três jeitos de o cliente mandar dados

De onde vêm os dados?

req.body

No corpo da requisição, em JSON. Usado para criar e atualizar.

```
POST /baralhos  
{ "titulo": "Node" }
```

req.params

Na própria rota, no lugar do **:nome**. Identifica um item.

```
GET /baralhos/2
```

req.query

Depois do ? na URL. Usado para buscas e filtros.

```
GET /flashcards?termo=node
```

req.body

```
app.post('/baralhos', (req, res) => {  
  const titulo = req.body.titulo  
  res.status(201).send({  
    mensagem: 'Dados recebidos pelo req.body',  
    titulo: titulo,  
  })  
})
```

No Postman

```
POST /baralhos { "titulo": "Node" }  
→ 201 { "mensagem": "Dados recebidos pelo req.body",  
        "titulo": "Node" }
```

req.params

```
app.get('/baralhos/:id', (req, res) => {  
  const id = req.params.id  
  res.status(200).send({  
    mensagem: 'Parâmetro recebido pelo req.params',  
    id: id,  
  })  
})
```

No Postman

```
GET /baralhos/2  
→ 200 { "mensagem": "Parâmetro recebido pelo req.params",  
        "id": "2" }
```

req.query

```
app.get('/flashcards', (req, res) => {  
  const termo = req.query.termo  
  res.status(200).send({  
    mensagem: 'Busca recebida pelo req.query',  
    termo: termo,  
  })  
})
```

No Postman

```
GET /flashcards?termo=node  
→ 200 { "mensagem": "Busca recebida pelo req.query",  
        "termo": "node" }
```



Cuidado: req.body chegou undefined?

Sem o `app.use(express.json())`, o Express **não lê** o JSON do body.

- Coloque o `express.json()` antes das rotas.
- No Postman, envie em Body > raw > JSON: é isso que coloca o header `Content-Type: application/json`.

```
app.use(express.json()) // antes das rotas!  
  
app.post('/baralhos', (req, res) => {  
  console.log(req.body) // sem o express.json(): undefined  
})
```



Cuidado: req.params é sempre texto

Em **GET /baralhos/2**, o **req.params.id** é a string **"2"**, não o número 2.

- Comparar com o id do objeto usando **===** dá false, e o item "não existe".
- Converta antes de buscar: **parseInt(req.params.id)**. O mesmo vale para o **req.query**.

```
// em GET /baralhos/2, req.params.id vale "2"
baralhos.find((b) => b.id === req.params.id)           // undefined
baralhos.find((b) => b.id === parseInt(req.params.id)) // encontra
```



Respondendo o cliente

res.status() e res.send()

```
app.post('/baralhos', (req, res) => {  
  const titulo = req.body.titulo  
  
  if (!titulo) {  
    return res.status(400).send({ message: 'O título é obrigatório.' })  
  }  
  
  const novo = { id: 3, titulo: titulo }  
  res.status(201).send({ message: 'Baralho criado!', baralho: novo })  
})
```

O `status()` define o código (sem ele, o padrão é 200). O `send()` com um objeto já envia JSON; o `res.json()` faz o mesmo de forma explícita.



Cuidado: responder duas vezes

Cada requisição recebe **uma única** resposta. A segunda dá erro: **Cannot set headers after they are sent.**

- Acontece quando o código responde o erro e continua executando até o próximo **res.send**.
- Por isso usamos **return res.status(...)**: o **return** encerra a função ali.

```
if (!titulo) {  
  res.status(400).send({ message: 'Título obrigatório' })  // falta o return!  
}  
res.status(201).send({ message: 'Criado!' })  // erro: segunda resposta
```



Cuidado: esqueceu de responder

Se a rota não chama `res.send`, o cliente fica esperando para sempre.

- No Postman, a requisição fica carregando até dar timeout.
- Toda rota, e todo caminho dentro dela (if, else), precisa terminar com uma resposta.

204 No Content: deu certo, sem corpo

```
app.delete('/baralhos/:id', (req, res) => {  
  const resultado = deletarBaralho(req.params.id)  
  if (resultado.error) {  
    return res.status(404).json({ message: resultado.error })  
  }  
  res.status(204).send() // sucesso, sem body  
})
```

No Postman

```
DELETE /baralhos/2  
→ 204 No Content (a resposta não tem body)
```

Use 204 quando não há nada para devolver. Quer mandar uma confirmação? Aí é 200 com JSON.



204 com mensagem: o que o cliente recebe?

```
app.delete('/baralhos/:id', (req, res) => {  
  // ... remove o baralho  
  res.status(204).json({ message: 'Baralho removido!' })  
})
```

Pense antes de testar: o que aparece no Postman?



Cuidado: o 204 não tem corpo

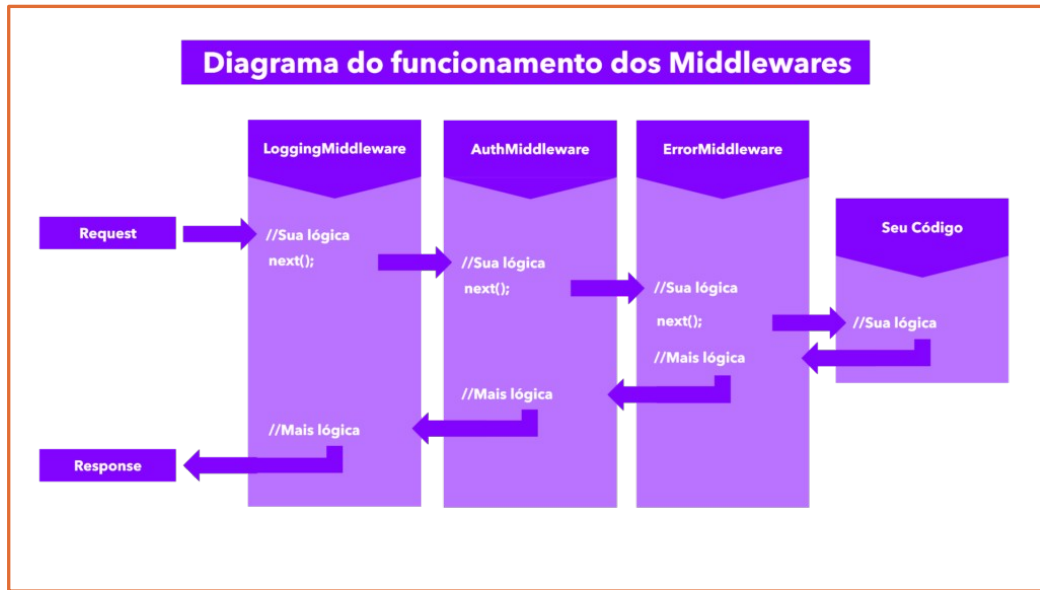
O Postman mostra **204** e o body **vazio**: o Express descarta o conteúdo de uma resposta 204.

- Não é erro, é a regra do HTTP: 204 significa "sem conteúdo".
- Mensagem para o usuário? Responda 200 com JSON. Nada a devolver?
`res.status(204).send()`.
- No frontend, não tente ler JSON de uma resposta 204: não há nada para ler.



Middlewares

O que é um middleware?



- Uma função que fica entre a chegada da requisição e a resposta final.
- Pode registrar logs, validar dados, verificar autenticação ou tratar erros.
- O `next()` passa a requisição adiante: para o próximo middleware ou para a rota.

Um middleware de log

```
app.use((req, res, next) => {  
  console.log(`${req.method} ${req.url}`)  
  next()    // sem isso, a requisição para aqui  
})  
  
app.get('/', (req, res) => {  
  res.send('Hello World!')  
})
```

Saída no terminal

```
GET /  
POST /baralhos
```

Middleware de erro

```
app.get('/', (req, res) => {  
  throw new Error('CÓDIGO QUEBROU')  
})  
  
// sempre com 4 parâmetros, e depois das rotas  
app.use((err, req, res, next) => {  
  console.error(err.stack)  
  res.status(500).send('Alguma coisa deu errado!')  
})
```

4 parâmetros → é o **err** na frente que faz o Express reconhecer um middleware de erro.

depois das rotas → ele recebe os erros que as rotas lançaram.

500 → o cliente recebe uma resposta clara, e o servidor não cai.

Rota que não existe: 404

```
// depois de todas as rotas, antes do middleware de erro
app.use((req, res) => {
  res.status(404).json({ message: `Rota ${req.method} ${req.url} não encontrada` })
})
```

Sem isso, o Express responde uma página HTML genérica. Com isso, o cliente recebe um **JSON** padronizado, como o resto da API.

PUT x PATCH

PUT

Substitui

Envia o recurso inteiro. O que não foi enviado pode ser apagado.

PATCH

Altera uma parte

Envia só os campos que mudaram. O resto fica como estava.

Na prática, muitas APIs usam o PUT de forma parcial, como a nossa. No Módulo 9, a atividade dos livros usa PATCH.



Validando os dados

Nunca confie no que o cliente manda

O que pode chegar no body



Campo faltando

O cliente esqueceu um campo obrigatório.

```
{ "resposta": "4" }
```



Tipo errado

Texto onde deveria vir um número.

```
{ "idBaralho": "abc" }
```



Texto vazio

Só espaços passam no !pergunta.

```
{ "pergunta": "  " }
```



Campo a mais

O cliente tenta escolher o que não devia.

```
{ "id": 99 }
```

Validar antes da lógica: a função recebe dados confiáveis, e o cliente recebe um 400 que explica o erro.

Validando no começo da rota

```
app.post('/flashcards', (req, res) => {  
  const { pergunta, resposta, idBaralho } = req.body  
  
  if (!pergunta || !resposta) {  
    return res.status(400).json({ message: 'Faltou pergunta ou resposta.' })  
  }  
  const id = parseInt(idBaralho)  
  if (Number.isNaN(id)) {  
    return res.status(400).json({ message: 'idBaralho inválido.' })  
  }  
  
  // daqui para baixo, os dados já foram conferidos  
  const resultado = adicionarFlashcard(pergunta, resposta, id)  
  if (resultado.error) {  
    return res.status(404).json({ message: resultado.error })  
  }  
  res.status(201).json({ flashcard: resultado.data })  
})
```

Cada if barra um problema e encerra com return. Passou por todos? Os dados estão conferidos.



Cuidado: !valor também barra o 0

Com `if (!quantidade)`, uma quantidade **0** é recusada: o 0 é falsy, como vimos no Módulo 2.

- O mesmo vale para **false** e para o texto vazio: às vezes, eles são valores válidos.
- Para saber se o campo veio, compare com **undefined**. Para saber se o valor serve, confira o tipo e o intervalo.

```
if (!quantidade)           // barra o 0 também: errado
if (quantidade === undefined) // o campo não veio
if (typeof quantidade !== 'number' || quantidade < 0) // valor inválido
```

Validação como middleware

```
function validarBaralho(req, res, next) {  
  const { titulo } = req.body  
  if (typeof titulo !== 'string' || titulo.trim() === '') {  
    return res.status(400).json({ message: 'O título é obrigatório.' })  
  }  
  next()  
}  
  
app.post('/baralhos', validarBaralho, (req, res) => { /* criar */ })  
app.put('/baralhos/:id', validarBaralho, (req, res) => { /* atualizar */ })
```

A mesma validação serve para o POST e o PUT, sem repetir código. No Módulo 9, o boilerplate faz isso com o validarCampos.



Flashcards como API

O mesmo sistema, agora acessado por rotas HTTP

Relembrando o sistema de flashcards



Estrutura

- Cada flashcard tem uma pergunta e uma resposta.
- Os flashcards são organizados em baralhos.



Requisitos

- Criar baralhos e flashcards.
- Buscar individualmente ou por baralho.
- Atualizar e remover.

Requisitos dos baralhos



Cadastro e busca

- Gerar um id para cada baralho.
- Título obrigatório; sem título, mensagem de erro.
- Listar todos os baralhos, com id e título.



Atualização e remoção

- Localizar o baralho pelo id antes de alterar.
- Remover um baralho remove os flashcards dele.
- Id inexistente: mensagem de erro.

Requisitos da API com Express



O servidor é criado com o Express e as requisições aceitam JSON.



As funcionalidades são acessadas por rotas: POST cadastra, GET lista e busca, PUT atualiza e DELETE remove.



Os status HTTP são adequados: 200, 201, 400 e 404.

Da estrutura antiga para a nova



Antes (Módulo 6)

- **index.js**: menu no terminal.
- Arquivos de baralhos e flashcards com as funções.



Agora (Módulo 7)

- **index.js**: servidor Express com as rotas.
- As mesmas funções, chamadas pelas rotas.

A lógica não muda. Só troca quem chama as funções: antes era o menu, agora é o cliente HTTP.

Os dados: dados.js

```
let baralhos = [  
  { id: 1, titulo: 'JavaScript' },  
  { id: 2, titulo: 'Matematica' },  
]  
  
let flashcards = [  
  { id: 1, pergunta: 'Quanto é 1+1?', resposta: '2', idBaralho: 2 },  
]  
  
export { baralhos, flashcards }
```

Cada flashcard guarda o idBaralho: o relacionamento 1:N do Módulo 6.

A lógica: adicionarBaralho

```
function adicionarBaralho(titulo) {  
  if (!titulo) {  
    return { error: 'O título é obrigatório.' }  
  }  
  let novoId = 1  
  if (baralhos.length > 0) {  
    novoId = baralhos[baralhos.length - 1].id + 1  
  }  
  const novoBaralho = { id: novoId, titulo: titulo }  
  baralhos.push(novoBaralho)  
  return { data: novoBaralho }  
}
```

A função devolve um objeto com error ou com data. Quem decide o status HTTP é a rota, não a função.

A rota: POST /baralhos

```
app.post('/baralhos', (req, res) => {  
  const titulo = req.body.titulo  
  const resultado = adicionarBaralho(titulo)  
  
  if (resultado.error) {  
    return res.status(400).send({ message: resultado.error })  
  }  
  res.status(201).send({  
    message: 'Baralho criado com sucesso!',  
    baralho: resultado.data,  
  })  
})
```

Erro de validação → 400. Criado → 201, devolvendo o baralho novo.

Testando: POST /baralhos

Requisição

POST /baralhos

```
{  
  "titulo": "Node.js"  
}
```

Resposta

201 Created

```
{  
  "message": "Baralho criado!",  
  "baralho": {  
    "id": 3,  
    "titulo": "Node.js"  
  }  
}
```

Teste também sem o título: a resposta deve ser **400** com a mensagem de erro.

GET e PUT

```
app.get('/baralhos', (req, res) => {  
  res.status(200).send(listarBaralho())  
})  
  
app.put('/baralhos/:id', (req, res) => {  
  const id = req.params.id  
  const titulo = req.body.titulo  
  const resultado = atualizarBaralho(id, titulo)  
  if (resultado.error) {  
    return res.status(404).send({ message: resultado.error })  
  }  
  res.status(200).send({ baralho: resultado.data })  
})
```

O PUT usa as duas fontes: o id vem do req.params e o título novo vem do req.body.

DELETE: removendo o baralho e os flashcards

```
function deletarBaralho(id) {  
  const idNum = parseInt(id)  
  const index = baralhos.findIndex((b) => b.id === idNum)  
  if (index === -1) return { error: 'Baralho não encontrado.' }  
  
  const removido = baralhos[index]  
  baralhos.splice(index, 1)  
  
  for (let i = flashcards.length - 1; i >= 0; i--) {  
    if (flashcards[i].idBaralho === idNum) flashcards.splice(i, 1)  
  }  
  return { data: removido }  
}
```

O laço anda de trás para frente: ao remover com splice, os itens seguintes mudam de posição, e um laço para frente pularia alguns.

Todas as rotas do sistema

Rota	O que faz
POST /baralhos	Cria um baralho
GET /baralhos	Lista os baralhos
PUT /baralhos/:id	Atualiza um baralho
DELETE /baralhos/:id	Remove o baralho e os flashcards dele
POST /flashcards	Cria um flashcard
GET /flashcards	Lista (ou busca) os flashcards
PUT /flashcards/:id	Atualiza um flashcard
DELETE /flashcards/:id	Remove um flashcard
GET /baralhos/:idBaralho/flashcards	Lista os flashcards de um baralho



Organizando as rotas

Dez rotas num arquivo só? O `express.Router()` separa por recurso.

routes/baralhos.routes.js

```
import { Router } from 'express'
import adicionarBaralho from '../adicionar_baralho.js'
import listarBaralho from '../listar_baralho.js'

const router = Router()

router.post('/', (req, res) => {
  const resultado = adicionarBaralho(req.body.titulo)
  if (resultado.error) {
    return res.status(400).json({ message: resultado.error })
  }
  res.status(201).json({ baralho: resultado.data })
})

router.get('/', (req, res) => {
  res.status(200).json(listarBaralho())
})

export default router
```

Router() → um "mini app" que só guarda rotas.

router.post('/') → o caminho é relativo: o prefixo vem depois, no index.js.

../ → o arquivo está na pasta routes: os imports sobem um nível, com o .js no final (Módulo 5).

Registrando as rotas no index.js

```
import express from 'express'
import baralhos from './routes/baralhos.routes.js'
import flashcards from './routes/flashcards.routes.js'

const app = express()
app.use(express.json())

app.use('/baralhos', baralhos)
app.use('/flashcards', flashcards)

app.listen(3000)
```

`app.use('/baralhos')`
→ todas as rotas do arquivo ganham esse prefixo.

`'/'` → vira `/baralhos`;
o `/:id` vira `/baralhos/:id`.

rota aninhada →
o `/:idBaralho/flashcards`
fica no arquivo de baralhos.



Cuidado: o prefixo não se repete

Com `router.get("/baralhos")` e `app.use("/baralhos", router)`, a rota vira `/baralhos/baralhos`. O GET `/baralhos` dá 404.

- O prefixo fica só no `app.use`. Dentro do arquivo de rotas, os caminhos começam em `/`.
- Deu 404 numa rota que existe? Confira o caminho completo: prefixo + caminho do router.

```
// routes/baralhos.routes.js
router.get('/', listar)           // certo: GET /baralhos
router.get('/baralhos', listar)  // errado: GET /baralhos/baralhos
```



Hora da Prática

Atividade Prática

Express

1. Crie a pasta `modulo07`.
2. Use o flashcards dos slides (ou do repositório) para entender o Express.
3. Copie os exercícios de CRUD do Módulo 6 para a pasta nova.
4. Adapte cada um para ser acessado por rotas, e não pelo terminal.
5. Teste todas as rotas no Postman, no Insomnia ou com cURL.



Dica: use `node --watch index.js` para o servidor reiniciar sozinho, e confira os status: 201 ao criar, 400 com dado inválido, 404 com id inexistente.

Lista de exercícios



Lista de código

Rotas GET, POST, PUT e DELETE, req.body, req.params e status: complete, ache o bug e preveja a resposta.



APIs aplicadas

Estudantes, Filmes e Livros: um CRUD REST completo em cada.



Desafio

Uma API de tarefas, escrita do zero.

No repositório: [modulo07/lista_de_exercicios/](#)

Material do módulo



Aponte a câmera do celular

[github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo07](https://github.com/nicolassmotta/curso-lions-dev/tree/main/modulo07)



conteudo/
teoria comentada e o cheat sheet do módulo



exercicios_slides/
os exercícios feitos em aula



lista_de_exercicios/
listas para praticar em casa



exercicios_resolvidos/
gabaritos, para conferir depois de tentar



Fim!

No Módulo 8, os dados saem da memória e vão para um banco de dados.